## COMPILER FLAGS USED:

`-std=c++17 -O3`, except for pt.2's multiple optimization levels.

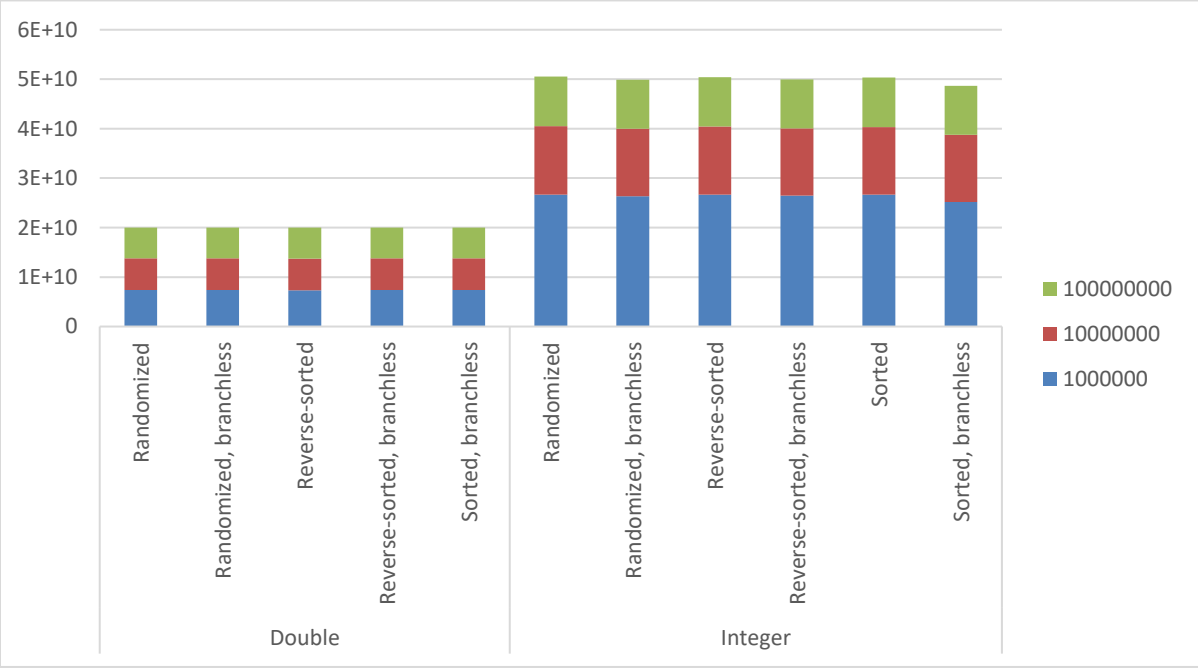
## Pt. 1 –

## Raw Data:

| N | Data_Type | Configuration | Min_ns | Max_ns | Avg_ns | Med_ns | Bytes_Per_Sec |
|---|---|---|---|---|---|---|---|
| 1000000 | Integer | Sorted | 148921 | 153434 | 150063 | 149822 | 26655400000 |
| 1000000 | Integer | Sorted, branchless | 150380 | 216882 | 158719 | 152310 | 25201600000 |
| 1000000 | Integer | Reverse-sorted | 148903 | 153496 | 150007 | 149715 | 26665400000 |
| 1000000 | Integer | Reverse-sorted, branchless | 150562 | 154994 | 151179 | 150745 | 26458700000 |
| 1000000 | Integer | Randomized | 149537 | 152583 | 149944 | 149638 | 26676500000 |
| 1000000 | Integer | Randomized, branchless | 151373 | 155065 | 152009 | 151720 | 26314100000 |
| 1000000 | Double | Sorted, branchless | 1087040 | 1090856 | 1087812 | 1087441 | 7354210000 |
| 1000000 | Double | Reverse-sorted | 1087232 | 1097703 | 1088950 | 1087499 | 7346520000 |
| 1000000 | Double | Reverse-sorted, branchless | 1087240 | 1090901 | 1087710 | 1087373 | 7354900000 |
| 1000000 | Double | Randomized | 1087276 | 1090507 | 1087758 | 1087487 | 7354580000 |
| 1000000 | Double | Randomized, branchless | 1087316 | 1090125 | 1087741 | 1087444 | 7354690000 |
| 10000000 | Integer | Sorted | 2890410 | 2952473 | 2925180 | 2926488 | 13674400000 |
| 10000000 | Integer | Sorted, branchless | 2925422 | 2986580 | 2950316 | 2950563 | 13557900000 |
| 10000000 | Integer | Reverse-sorted | 2861517 | 2938252 | 2908849 | 2908018 | 13751100000 |
| 10000000 | Integer | Reverse-sorted, branchless | 2910153 | 2974241 | 2944879 | 2946515 | 13582900000 |
| 10000000 | Integer | Randomized | 2872892 | 2901269 | 2886418 | 2885719 | 13858000000 |
| 10000000 | Integer | Randomized, branchless | 2904229 | 2962897 | 2931225 | 2926610 | 13646200000 |
| 10000000 | Double | Sorted, branchless | 12372293 | 12535506 | 12455885 | 12456053 | 6422670000 |
| 10000000 | Double | Reverse-sorted | 12394526 | 12564524 | 12488019 | 12509048 | 6406140000 |
| 10000000 | Double | Reverse-sorted, branchless | 12359862 | 12527016 | 12439178 | 12416122 | 6431290000 |
| 10000000 | Double | Randomized | 12377057 | 12569615 | 12455417 | 12404297 | 6422910000 |
| 10000000 | Double | Randomized, branchless | 12417184 | 12486684 | 12441255 | 12424900 | 6430220000 |
| 100000000 | Integer | Sorted | 39954285 | 40104593 | 39998756 | 39991011 | 10000300000 |
| 100000000 | Integer | Sorted, branchless | 40254677 | 40334128 | 40288438 | 40286867 | 9928410000 |
| 100000000 | Integer | Reverse-sorted | 39958951 | 40054671 | 39999263 | 39994970 | 10000200000 |
| 100000000 | Integer | Reverse-sorted, branchless | 40290651 | 40357125 | 40312081 | 40309833 | 9922580000 |
| 100000000 | Integer | Randomized | 39957905 | 40582076 | 40062964 | 40004498 | 9984280000 |
| 100000000 | Integer | Randomized, branchless | 40290743 | 40362416 | 40325462 | 40326247 | 9919290000 |
| 100000000 | Double | Sorted, branchless | 127687967 | 128884293 | 128112084 | 127833227 | 6244530000 |

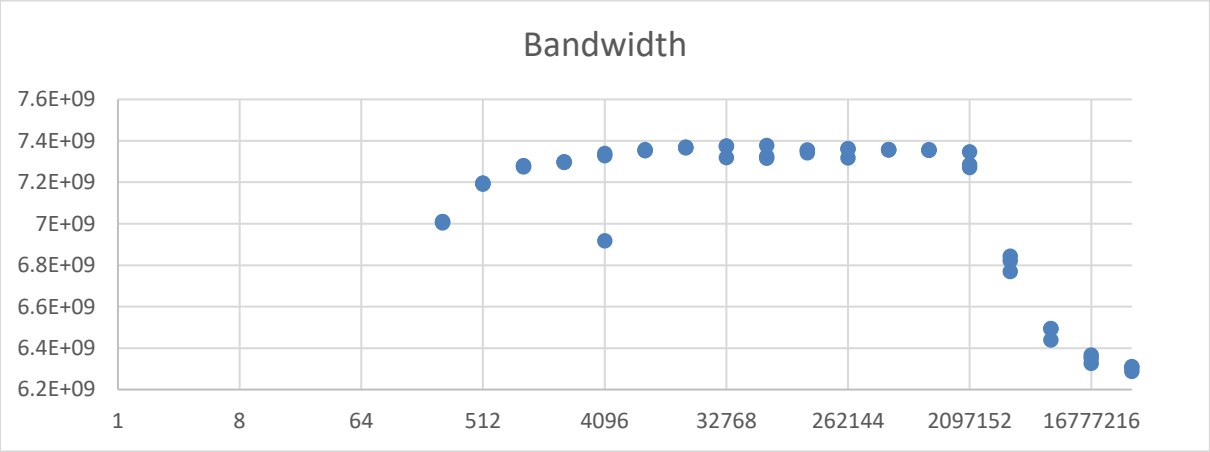
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 100000000 | Double | Reverse-sorted | 12774403 4 | 12886090 8 | 12826476 2 | 12837889 6 | 6237100000 |
| 100000000 | Double | Reverse-sorted, branchless | 12742704 0 | 12890845 6 | 12826958 4 | 12842906 9 | 6236860000 |
| 100000000 | Double | Randomized | 12770139 4 | 12866992 2 | 12793378 3 | 12779151 2 | 6253230000 |
| 100000000 | Double | Randomized, branchless | 12747465 9 | 12967216 7 | 12844152 2 | 12820539 9 | 6228520000 |

The graph measures the bytes per second (along the Y axis) for each tested N value.




As you can see, integers perform significantly faster than doubles, likely owing to their reduced number of bytes. Additionally, the order of the list had no influence on performance. The branch / branchless variants had no difference in performance, either – I think the compiler likely optimized it out.

On this chart, the X-axis is the size of the array (scaled logarithmically, base two), while the Y-axis is the throughput.



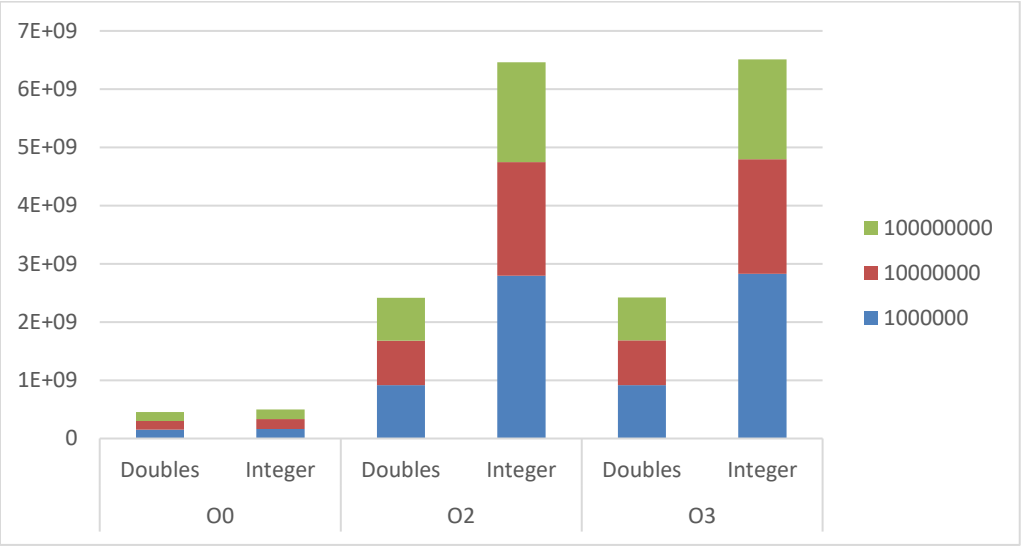

I wasn't able to mark the specific point on the graph (curse you Excel!) but you can see the throughput drops off sharply after 2^21 – which makes sense, because that's the delineation between the L2 and L3 caches!

**Pt. 2 –**

Raw Data:

| N | Opt. Lvl | Data_Type | Min_ns | Max_ns | Avg_ns | Med_ns | Elements_Per_Sec |
|---|---|---|---|---|---|---|---|
| 1000000 | O0 | Integer | 6008284 | 6016300 | 6012342 | 6012109 | 166325000 |
| 1000000 | O0 | Doubles | 6568597 | 6577150 | 6573246 | 6573074 | 152089000 |
| 10000000 | O0 | Integer | 60137399 | 60223365 | 60172278 | 60163863 | 166189000 |
| 10000000 | O0 | Doubles | 66455344 | 66562491 | 66492002 | 66484455 | 150313000 |
| 100000000 | O0 | Integer | 601350226 | 602849035 | 601746038 | 601546277 | 166183000 |
| 100000000 | O0 | Doubles | 663385283 | 664330415 | 663755972 | 663463453 | 150729000 |
| 1000000 | O2 | Integer | 352327 | 382917 | 357690 | 355151 | 2795710000 |
| 1000000 | O2 | Doubles | 1088749 | 1095055 | 1089686 | 1089093 | 916807000 |
| 10000000 | O2 | Integer | 5097642 | 5150523 | 5127411 | 5130941 | 1950300000 |
| 10000000 | O2 | Doubles | 12832452 | 13245314 | 13061255 | 13107375 | 762887000 |
| 100000000 | O2 | Integer | 58001590 | 58362714 | 58213415 | 58210671 | 1717820000 |
| 100000000 | O2 | Doubles | 133334936 | 137821507 | 136288402 | 137077058 | 734617000 |
| 1000000 | O3 | Integer | 352579 | 357420 | 353727 | 352864 | 2827040000 |
| 1000000 | O3 | Doubles | 1087313 | 1090333 | 1087950 | 1087552 | 919116000 |
| 10000000 | O3 | Integer | 5050615 | 5119477 | 5084633 | 5085480 | 1966710000 |
| 10000000 | O3 | Doubles | 12824598 | 13212902 | 13024699 | 13010885 | 767753000 |
| 100000000 | O3 | Integer | 57968601 | 58463592 | 58239898 | 58257464 | 1717040000 |
| 100000000 | O3 | Doubles | 133382026 | 137646199 | 136178702 | 136991997 | 732889000 |

On this chart, the Y-axis represents elements per second.




You can see, a similar pattern exists with the doubles / integers as before, though it is significantly less pronounced at O0 optimization.

Expectedly, the optimizations make a major difference in performance.

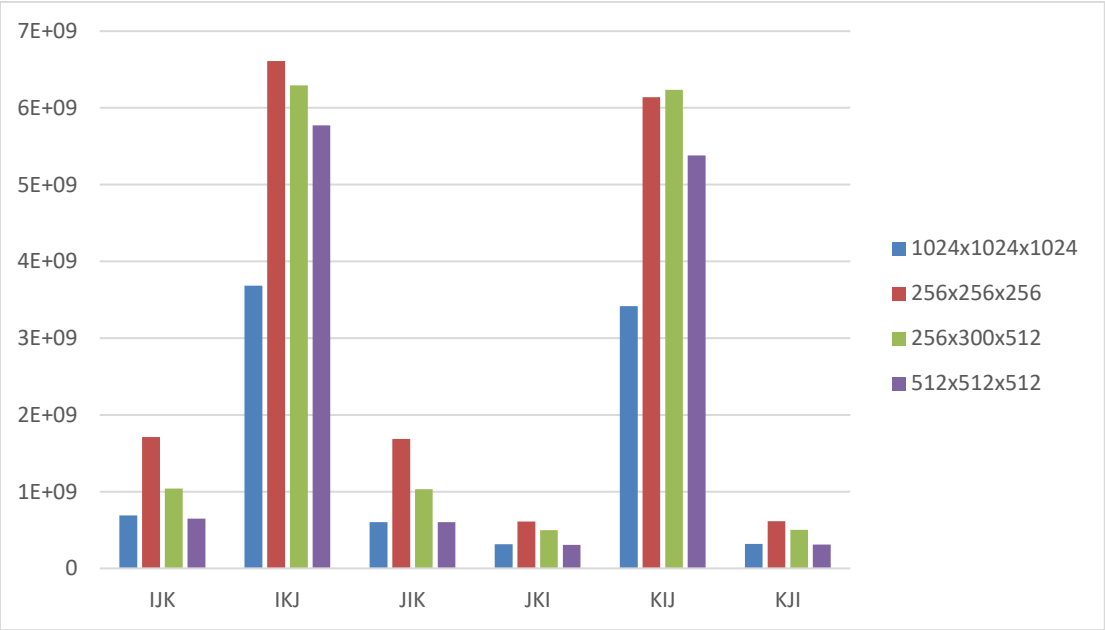
**Pt. 3 –**

Raw data:

| Size | Loop_Order | Min_ns | Max_ns | Avg_ns | Med_ns | FLOPS |
|---|---|---|---|---|---|---|
| 256x256x256 | IJK | 19597610 | 19631059 | 19618336 | 19618192 | 1710360000 |
| 256x256x256 | IKJ | 5057836 | 5096856 | 5074480 | 5065775 | 6612390000 |
| 256x256x256 | JIK | 19899736 | 19938554 | 19913259 | 19908550 | 1685030000 |
| 256x256x256 | JKI | 55049660 | 55113151 | 55074519 | 55067250 | 609255000 |
| 256x256x256 | KIJ | 5462785 | 5470107 | 5466685 | 5466250 | 6137980000 |
| 256x256x256 | KJI | 54639756 | 54676243 | 54652058 | 54650051 | 613965000 |
| 512x512x512 | IJK | 413892404 | 413990583 | 413933051 | 413929229 | 648500000 |
| 512x512x512 | IKJ | 46022968 | 46838611 | 46491117 | 46485847 | 5773910000 |
| 512x512x512 | JIK | 443493895 | 448474806 | 445061933 | 444825648 | 603142000 |
| 512x512x512 | JKI | 858442065 | 880376824 | 870914119 | 871227550 | 308223000 |
| 512x512x512 | KIJ | 48307279 | 51716252 | 49876374 | 49711526 | 5382020000 |
| 512x512x512 | KJI | 857212457 | 861376477 | 859220335 | 859045724 | 312417000 |
| 1024x1024x1024 | IJK | 3103531521 | 3123403629 | 3113414459 | 3113981918 | 689752000 |
| 1024x1024x1024 | IKJ | 577617606 | 588993387 | 583116279 | 583598218 | 3682770000 |
| 1024x1024x1024 | JIK | 3480312419 | 3666175497 | 3559924535 | 3578691961 | 603239000 |
| 1024x1024x1024 | JKI | 6766851370 | 6869783765 | 6812180439 | 6812445787 | 315242000 |
| 1024x1024x1024 | KIJ | 627030410 | 630415989 | 628157013 | 628110230 | 3418710000 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 1024x1024x1024 | KJI | 6688528831 | 6987581296 | 6782450919 | 6793646497 | 316624000 |
| 256x300x512 | IJK | 75625881 | 76330601 | 75714059 | 75646951 | 1038690000 |
| 256x300x512 | IKJ | 12479567 | 12512817 | 12491282 | 12487155 | 6295850000 |
| 256x300x512 | JIK | 76260617 | 76307541 | 76281206 | 76276517 | 1030960000 |
| 256x300x512 | JKI | 157817745 | 157913332 | 157854583 | 157852751 | 498200000 |
| 256x300x512 | KIJ | 12579776 | 12647543 | 12613553 | 12620388 | 6234820000 |
| 256x300x512 | KJI | 155945003 | 156028142 | 155975096 | 155967622 | 504204000 |

On this chart, the Y axis represents the FLOPs of the matrix multiplication.




As you can see, the order of iteration has a major impact on performance! The IKJ and KIJ systems are especially slow, while the JKI and KJI systems are comparatively lightning-fast. It seems the ones that iterate `j` last are the slowest, while the ones that iterate `i` last are the quickest. This makes sense, given the index order or the matrices.

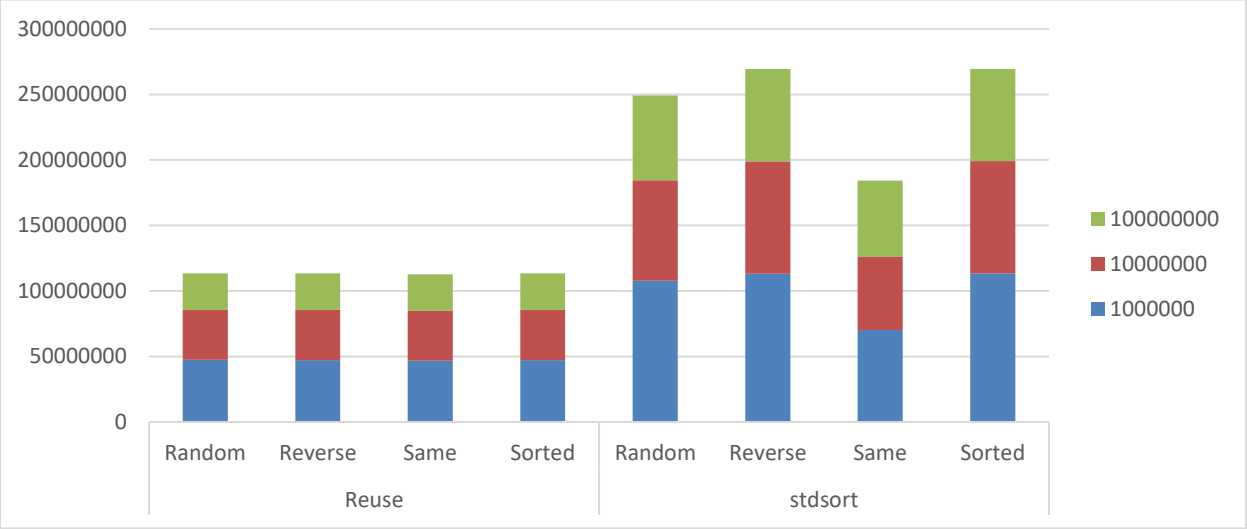
**Pt. 4 –**

Unfortunately, I don't have much too show for this one – the reallocation-every-time sort method wasn't able to produce results in time! I left it running for three hours, and it produced no results. Which, I suppose, goes to show how incredibly, incredibly slow it is.

Raw Data:

| N | Data_Pattern | Algorithm | Min_ns | Max_ns | Avg_ns | Med_ns | Elements_Per_Sec |
|---|---|---|---|---|---|---|---|
| 1000000 | Same | stdsort | 14250220 | 14269723 | 14261591 | 14264848 | 70118400 |
| 1000000 | Same | Reuse | 21327783 | 22202745 | 21425069 | 21336142 | 46674300 |
| 1000000 | Sorted | stdsort | 8770186 | 8841600 | 8811394 | 8814136 | 113489000 |
| 1000000 | Sorted | Reuse | 21113060 | 21133243 | 21123159 | 21123170 | 47341400 |
| 1000000 | Reverse | stdsort | 8821216 | 8850779 | 8835765 | 8834726 | 113176000 |
| 1000000 | Reverse | Reuse | 21113857 | 21156311 | 21128411 | 21123967 | 47329600 |
| 1000000 | Random | stdsort | 9274812 | 9293220 | 9281974 | 9281070 | 107736000 |
| 1000000 | Random | Reuse | 21111975 | 21128484 | 21118777 | 21117290 | 47351200 |
| 10000000 | Same | stdsort | 177686872 | 177881266 | 177779708 | 177783443 | 56249400 |
| 10000000 | Same | Reuse | 261540342 | 262167779 | 261965306 | 262026703 | 38173000 |
| 10000000 | Sorted | stdsort | 116424348 | 116899614 | 116625844 | 116591170 | 85744300 |
| 10000000 | Sorted | Reuse | 261591056 | 262886694 | 261958078 | 261857774 | 38174000 |
| 10000000 | Reverse | stdsort | 116416595 | 117175337 | 116743197 | 116663861 | 85658100 |
| 10000000 | Reverse | Reuse | 261465427 | 261933353 | 261696804 | 261706751 | 38212200 |
| 10000000 | Random | stdsort | 130266206 | 130453459 | 130349989 | 130342449 | 76716500 |
| 10000000 | Random | Reuse | 261335542 | 262260609 | 261799726 | 261792414 | 38197100 |
| 100000000 | Same | stdsort | 1725446823 | 1727597598 | 1726204001 | 1725892226 | 57930600 |
| 100000000 | Same | Reuse | 3594875012 | 3618660984 | 3605457971 | 3606658537 | 27735700 |
| 100000000 | Sorted | stdsort | 1419666722 | 1432573613 | 1424469989 | 1424097186 | 70201500 |
| 100000000 | Sorted | Reuse | 3593877050 | 3599419403 | 3595564284 | 3595101935 | 27812000 |
| 100000000 | Reverse | stdsort | 1415248599 | 1420743746 | 1417885622 | 1417694123 | 70527600 |
| 100000000 | Reverse | Reuse | 3593314719 | 3621179920 | 3599883219 | 3594998591 | 27778700 |
| 100000000 | Random | stdsort | 1513168335 | 1571322667 | 1544315603 | 1542491860 | 64753600 |
| 100000000 | Random | Reuse | 3592785884 | 3594665502 | 3593905675 | 3594084021 | 27824900 |

Elements processed per second are represented along the Y axis, and the category on the X.




As expected, the built-in `std::sort` can process elements much faster than my humble, ad-hoc merge sort implementation…

It seems that, on my implementation, all array arrangements are approximately the same. On `std::sort`'s version, the "same" ordering runs significantly faster. Interestingly, the "sorted" ordering doesn't seem particularly faster on `std::sort`. I find that rather odd…

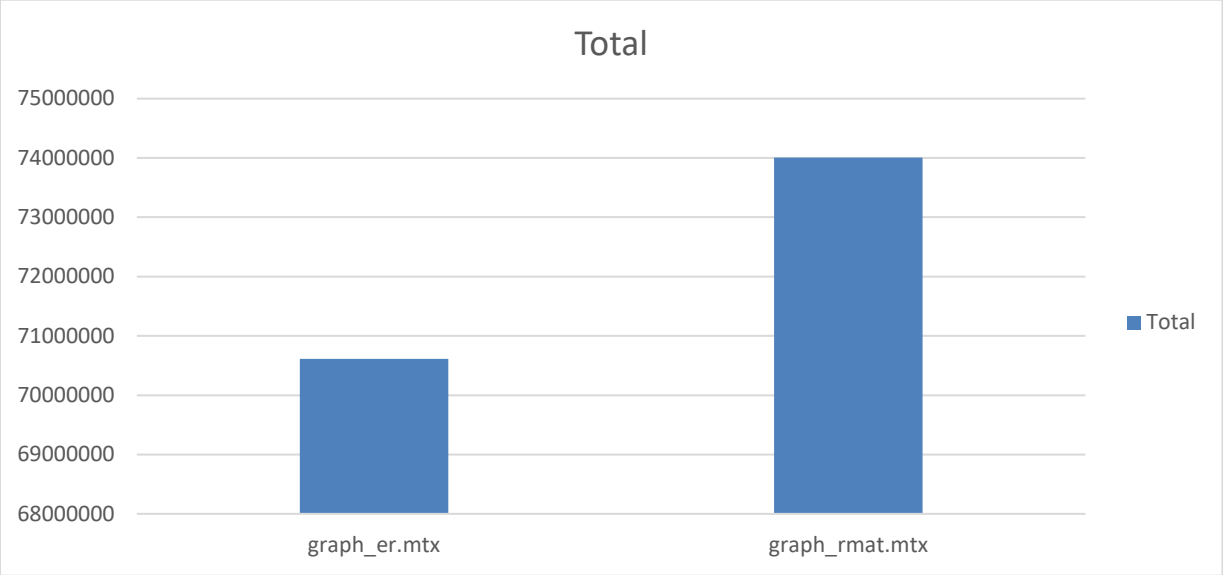
**Pt. 5 –**

Raw Data:

| File | Start_Node | Levels | Percent_Reached | Edges_Inspected | TEPS |
|---|---|---|---|---|---|
| graph_er.mtx | 572998 | 8 | 1 | 16772918 | 66719300 |
| graph_er.mtx | 803940 | 8 | 1 | 16772918 | 70353900 |
| graph_er.mtx | 442417 | 8 | 1 | 16772918 | 71444300 |
| graph_er.mtx | 265001 | 8 | 1 | 16772918 | 70571400 |
| graph_er.mtx | 451940 | 8 | 1 | 16772918 | 71413200 |
| graph_er.mtx | 275846 | 8 | 1 | 16772918 | 70675000 |
| graph_er.mtx | 704493 | 8 | 1 | 16772918 | 71439800 |
| graph_er.mtx | 732572 | 8 | 1 | 16772918 | 70577700 |
| graph_er.mtx | 829143 | 8 | 1 | 16772918 | 70643100 |
| graph_er.mtx | 46367 | 8 | 1 | 16772918 | 71523100 |
| graph_er.mtx | 529602 | 8 | 1 | 16772918 | 71406300 |
| graph_er.mtx | 524551 | 8 | 1 | 16772918 | 70618600 |
| graph_er.mtx | 1025317 | 8 | 1 | 16772918 | 70460600 |
| graph_er.mtx | 1018952 | 8 | 1 | 16772918 | 70635600 |
| graph_er.mtx | 644128 | 8 | 1 | 16772918 | 70652200 |
| graph_er.mtx | 896930 | 8 | 1 | 16772918 | 70628600 |
| graph_rmat.mtx | 572998 | 7 | 1 | 16777088 | 71475000 |
| graph_rmat.mtx | 803940 | 7 | 1 | 16777088 | 74316800 |
| graph_rmat.mtx | 442417 | 7 | 1 | 16777088 | 74218000 |
| graph_rmat.mtx | 265001 | 7 | 1 | 16777088 | 73629000 |

| | | | | | |
|---|---|---|---|---|---|
| graph_rmat.mtx | 451940 | 7 | 1 | 16777088 | 74228900 |
| graph_rmat.mtx | 275846 | 6 | 1 | 16777088 | 73597700 |
| graph_rmat.mtx | 704493 | 7 | 1 | 16777088 | 74228600 |
| graph_rmat.mtx | 732572 | 7 | 1 | 16777088 | 74284800 |
| graph_rmat.mtx | 829143 | 7 | 1 | 16777088 | 74257800 |
| graph_rmat.mtx | 46367 | 6 | 1 | 16777088 | 74305700 |
| graph_rmat.mtx | 529602 | 6 | 1 | 16777088 | 74247800 |
| graph_rmat.mtx | 524551 | 7 | 1 | 16777088 | 74174900 |
| graph_rmat.mtx | 1025317 | 6 | 1 | 16777088 | 74292300 |
| graph_rmat.mtx | 1018952 | 7 | 1 | 16777088 | 74266900 |
| graph_rmat.mtx | 644128 | 7 | 1 | 16777088 | 74339600 |
| graph_rmat.mtx | 896930 | 7 | 1 | 16777088 | 74269800 |

And the graph. TEPS on the Y, the graph file across the X.




It seems the r-mat graph traversal is significantly faster. How fun! I can only wonder why… I'm no graph theorist. I'll leave that to the professionals.

Excel, in Microsoft's infinite wisdom, won't let me create my own box-and-whisper plot – so, in its stead, I'll just dump a portion of the raw data here –

For the RMAT graph, the TEPS totals read:

```
MIN: 7.1475e+07

MEDIAN: 7.42578e+07

MAX: 7.43396e+07
```

And for the ER graph:

MIN: 6.67193e+07

MEDIAN: 7.06431e+07

MAX: 7.15231e+07